

VMCraft Engine Deep Dive

Pure Python Conversion Without libguestfs

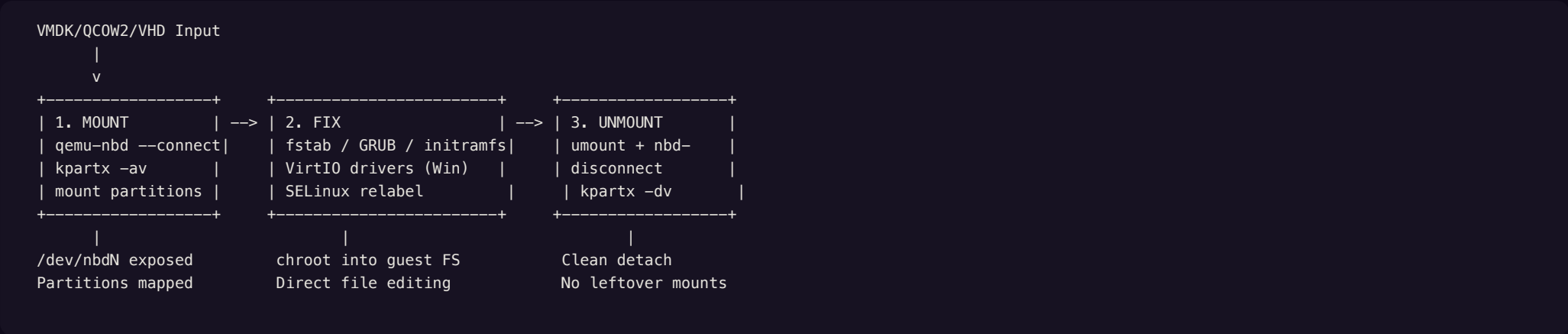
VMCraft is hyper2kvm's default conversion backend. It uses qemu-nbd for block device mounting, direct filesystem access for offline fixes, and chroot for initramfs regeneration. No supermin appliance, no 500MB dependency chain, no appliance boot delay. Just Python + qemu-nbd.

qemu-nbd | chroot | fstab | GRUB | VirtIO | Windows Registry | ~5MB footprint

The Mount → Fix → Unmount Pipeline

VMCraft's three-phase approach to offline VM repair.

Pipeline Overview



Phase 1: Mount — qemu-nbd Block Device Exposure

qemu-nbd Connection

- `modprobe nbd max_part=16` — load NBD kernel module
- `qemu-nbd --connect=/dev/nbd0 --format=vmdk disk.vmdk`
- Supports VMDK (monolithic/split), QCOW2, VHD, VHDX, raw
- Read-write access to guest disk as a local block device
- VMDK descriptor files auto-resolved (sparse/flat extents)
- No image conversion needed — operates on source format directly

Partition Discovery

- `kpartx -av /dev/nbd0` — creates `/dev/mapper/nbd0p1, nbd0p2...`
- Handles MBR and GPT partition tables automatically
- LVM detection: `vgscan + vgchange -ay` to activate VGs
- Root partition identified by filesystem label, UUID, or content heuristics
- EFI System Partition (ESP) detected for UEFI boot repair
- Swap partitions identified and skipped during mount

Phase 2: Fix — Offline Guest Filesystem Repair

fstab Repair with blkid

- VMware device names (`sda`) become VirtIO names (`vda`)
- `blkid /dev/mapper/nbd0p*` reads UUIDs from actual partitions
- Rewrites `/etc/fstab: /dev/sda1 → UUID=xxxx-xxxx`

GRUB Configuration

- Detects GRUB2 (`grub.cfg`) and legacy GRUB (`menu.lst`)
- Rewrites `root=` kernel parameter: `root=/dev/sda2 → root=UUID=...`
- Adds VirtIO modules to `GRUB_PRELOAD_MODULES` if needed

- Handles ext4, xfs, btrfs, swap entries
- Preserves mount options, dump/fsck fields
- Backs up original fstab before modification

- Regenerates grub.cfg via chroot: `grub2-mkconfig -o /boot/grub2/grub.cfg`
- UEFI: updates EFI/fedora/grub.cfg or EFI/redhat/grub.cfg
- Handles BLS (Boot Loader Spec) entries on Fedora/RHEL 8+

Initramfs Rebuild via chroot

```
# VMcraft chroot sequence for initramfs regeneration
mount --bind /dev /mnt/guest/dev
mount --bind /proc /mnt/guest/proc
mount --bind /sys /mnt/guest/sys

# Detect distro and kernel version
KVER=$(ls /mnt/guest/lib/modules/ | sort -V | tail -1)

# RHEL/Fedora: dracut
chroot /mnt/guest dracut --force --add-drivers "virtio_blk virtio_net virtio_pci virtio_scsi" \
    /boot/initramfs-${KVER}.img ${KVER}

# Debian/Ubuntu: update-initramfs
chroot /mnt/guest update-initramfs -u -k ${KVER}

# Cleanup bind mounts
umount /mnt/guest/{sys,proc,dev}
```

Windows Migration: Registry & VirtIO

VMCraft handles Windows VMs without requiring a running Windows instance.

Windows Registry Hive Loading

Offline Registry Editing

- Mount NTFS partition: `ntfs-3g /dev/mapper/nbd0p2 /mnt/guest`
- Load SYSTEM hive: `hivexregedit --merge` or Python hivex bindings
- Navigate to `HKLM\SYSTEM\CurrentControlSet\Services`
- Set VirtIO driver services to `Start=0` (boot start)
- Services: `viostor`, `vioscsi`, `vionet`, `balloon`, `vioserial`
- Remove VMware Tools services: `vmtoolsd`, `vmhgfs`, `VMCI`

VirtIO Driver Extraction from ISO

- Downloads or uses local `virtio-win.iso` (Red Hat signed drivers)
- Mounts ISO: `mount -o loop virtio-win.iso /mnt/virtio`
- Detects Windows version from registry: Win10, Win11, 2019, 2022
- Copies matching drivers: `/mnt/virtio/amd64/w10/viostor.sys`
- Installs `.inf` files into driver store: `Windows\INF\oem*.inf`
- Creates `CriticalDeviceDatabase` entries for boot-time loading

Windows VirtIO Registry Entries (Created by VMCraft)

```
; VirtIO SCSI boot driver – prevents BSOD 0x7B (INACCESSIBLE_BOOT_DEVICE)
[HKLM\SYSTEM\CurrentControlSet\Services\viostor]
"Type"=dword:00000001
"Start"=dword:00000000      ; 0 = Boot start (loaded by bootloader)
"ErrorControl"=dword:00000001
"Group"="SCSI miniport"
"ImagePath"="system32\drivers\viostor.sys"

; VirtIO Network – prevents "No network adapters" after migration
[HKLM\SYSTEM\CurrentControlSet\Services\netkvm]
"Type"=dword:00000001
"Start"=dword:00000003      ; 3 = Manual (PnP will start it)
"ImagePath"="system32\drivers\netkvm.sys"

; Critical Device Database – maps PCI VEN/DEV to driver
[HKLM\SYSTEM\CurrentControlSet\Control\CriticalDeviceDatabase\pci#ven_1af4&dev_1001]
"Service"="viostor"
"ClassGUID"="{4D36E97B-E325-11CE-BFC1-08002BE10318}"
```

VMCraft vs libguestfs: Architecture Comparison

Aspect

VMCraft (hyper2kvm default)

libguestfs

How it works	qemu-nbd exposes disk as /dev/nbdN, mount directly on host	Boots a supermin appliance (mini Linux VM), accesses disk inside appliance
Disk footprint	~5 MB (qemu-nbd binary)	~500 MB (supermin appliance + kernel + initrd)
Startup time	Instant (~100ms for nbd connect)	15-45 seconds (appliance boot)
Dependencies	qemu-utils, kpartx, Python 3	libguestfs, supermin, hivex, libvirt, kernel, febootstrap
Air-gap install	3 RPMs or DEBs	30+ RPMs, supermin rebuild may need network
Root required	Yes (nbd, mount, chroot)	No (runs in appliance VM)
SELinux compat	Works with enforcing (relabels guest)	Appliance may conflict with host SELinux
Windows support	Full VirtIO injection + registry editing	virt-v2v handles Windows (different tool)
Performance	Native I/O speed (direct mount)	Slightly slower (virtio inside appliance)
Debugging	Standard Linux tools (ls, cat, strace)	Must debug inside appliance (guestfish shell)

Complete Conversion Flow

Step-by-step walkthrough of a Linux VM conversion using VMCraft.

Example: Converting a RHEL 8 VMDK to KVM

```
$ h2kvmctl convert --source disk.vmdk --output /var/lib/libvirt/images/ --format qcow2 \
  --regen-initramfs --emit-domain-xml

[1/8] Validating source: disk.vmdk (VMDK, 40GB, thin provisioned)
[2/8] Converting VMDK -> QCOW2: qemu-img convert -p -f vmdk -O qcow2 -c disk.vmdk output.qcow2
      Progress: [=====] 100% 38.2 GB @ 245 MB/s
[3/8] Connecting qemu-nbd: /dev/nbd0 -> output.qcow2
[4/8] Discovering partitions: nbd0p1 (EFI, 600MB), nbd0p2 (xfs, 39.4GB)
[5/8] Mounting root: /dev/mapper/nbd0p2 -> /tmp/hyper2kvm-abc123/mnt
      Mounting EFI: /dev/mapper/nbd0p1 -> /tmp/hyper2kvm-abc123/mnt/boot/efi
[6/8] Applying fixes:
      - fstab: /dev/sda2 -> UUID=a1b2c3d4-e5f6-7890-abcd-ef1234567890
      - fstab: /dev/sda1 -> UUID=ABCD-1234 (vfat)
      - GRUB: root=/dev/sda2 -> root=UUID=a1b2c3d4-e5f6-7890-abcd-ef1234567890
      - initramfs: adding virtio_blk virtio_net virtio_pci virtio_scsi
      - dracut: regenerating /boot/initramfs-4.18.0-513.el8.x86_64.img
      - SELinux: touching /.autorelabel
[7/8] Unmounting: nbd0p2, nbd0p1, nbd-disconnect
[8/8] Generating libvirt XML: output.xml

Conversion complete: output.qcow2 (14.2 GB on disk, 40 GB virtual)
```

Code Architecture: Key Python Modules

vmcraft/mount.py

- NbdManager — context manager for qemu-nbd lifecycle
- Finds next free /dev/nbdN (checks /sys/block/nbdN/size)
- connect(path, fmt) — qemu-nbd --connect with retry
- disconnect() — qemu-nbd --disconnect with verification
- Timeout: waits up to 10s for /dev/nbdNp* to appear
- Cleanup: registered with atexit for crash safety

vmcraft/partitions.py

- discover_partitions(nbd_dev) — kpartx + blkid probe
- Returns list of Partition objects with UUID, fstype, label, size
- find_root(partitions) — heuristic: largest ext4/xfs/btrfs
- find_efi(partitions) — vfat partition with EFI flag
- LVM: scans for LV paths under /dev/mapper/vgname-lvname
- Handles multi-disk VMs by iterating nbd0, nbd1, ...

vmcraft/fixer.py

vmcraft/convert.py

- `fix_fstab(root_mount)` — UUID-based rewrite
- `fix_grub(root_mount)` — kernel cmdline + grub.cfg regen
- `rebuild_initramfs(root_mount, kver)` — chroot dracut/initramfs
- `inject_virtio_windows(root_mount)` — registry + driver copy
- `set_selinux_relabel(root_mount)` — touch /.autorelabel
- `remove_vmware_tools(root_mount)` — cleanup VMware artifacts

- `qemu_img_convert(src, dst, fmt, compress)` — format conversion
- Progress parsing: reads qemu-img stdout for percentage
- Supports: vmdk, qcow2, vhd, vhdx, raw output formats
- Compression: -c flag for qcow2 (zlib, ~40% size reduction)
- Checksum: SHA256 of output image for verification
- Thin provisioning: qcow2 inherently thin (sparse allocation)

Error Handling & Recovery

Crash Safety: VMCraft registers cleanup handlers with Python's `atexit` module. If the process crashes or is interrupted (Ctrl+C), the cleanup sequence runs: `umount -l` (lazy unmount), `kpartx -dv` (remove mappings), `qemu-nbd --disconnect` (release NBD). LVM volume groups are deactivated with `vgchange -an`. Temporary directories under `/tmp/hyper2kvm-*` are removed. This prevents stale mounts and NBD device leaks.

Advanced VMCraft Features

Beyond basic conversion: OVA extraction, multi-disk VMs, and performance tuning.

OVA/OVF Processing Pipeline

OVA Extraction

- OVA = tar archive containing .ovf + .vmdk + .mf (manifest)
- VMCraft streams extraction: reads tar members without full unpack
- Validates SHA256 checksums from .mf manifest file
- Parses .ovf XML for VM config: CPU, RAM, disk controllers
- Maps OVF hardware to libvirt XML: `VirtualSCSI` → `virtio-scsi`
- Handles split VMDKs: reassembles descriptor + extent files

Multi-Disk VM Handling

- Each VMDK gets its own `/dev/nbdN` (nbd0, nbd1, nbd2...)
- Root disk identified from OVF boot order or partition content
- Data disks: converted and attached to libvirt XML as `vdb`, `vdc`...
- Cross-disk LVM: VGs spanning multiple PVs detected and activated
- Independent `fstab` entries for each data disk
- Max 16 NBD devices (configurable via `nbd` module parameter)

Performance Characteristics

Operation	Time (40GB disk)	Bottleneck	Notes
<code>qemu-img convert</code> (VMDK→QCOW2)	2-3 min	Disk I/O	NVMe source: ~250 MB/s, SATA: ~150 MB/s
<code>qemu-nbd connect</code>	<1 sec	Kernel	NBD module load + device setup
<code>kpartx</code> + mount	<2 sec	Kernel	Partition table scan + filesystem mount
<code>fstab</code> + GRUB fix	<1 sec	CPU	Text parsing + <code>blkid</code> lookup
initramfs rebuild (dracut)	15-45 sec	CPU/Disk	Kernel module collection + <code>cpio</code> archive
Unmount + disconnect	<2 sec	Kernel	Sync + unmount + NBD release
Total (Linux VM)	3-5 min	Disk I/O	Dominated by <code>qemu-img convert</code>
Total (Windows VM)	4-7 min	Disk I/O	+1-2 min for VirtIO driver copy + registry

Supported Guest Operating Systems

Linux (Full Support)

Windows (Full Support)

- RHEL / CentOS / Rocky / Alma 7, 8, 9
- Fedora 36+
- Ubuntu 18.04, 20.04, 22.04, 24.04
- Debian 10, 11, 12
- SLES / openSUSE 15+
- VMware Photon OS 3, 4, 5

- Windows Server 2016, 2019, 2022, 2025
- Windows 10, 11 (Pro, Enterprise)
- VirtIO drivers: viostor, vioscsi, netkvm, balloon
- Automatic BSOD prevention (0x7B, 0x5C)
- VMware Tools removal
- Hyper-V enlightenments preserved

VMCraft: Simple, Fast, Reliable

No appliance boot. No 500MB dependency chain. No supermin.

Mount the disk, fix the files, unmount. Pure Python + qemu-nbd.

~5MB footprint. 3-5 minute conversions. Works in air-gapped environments.